



JOHNS HOPKINS
UNIVERSITY

EN.601.422 / EN.601.622

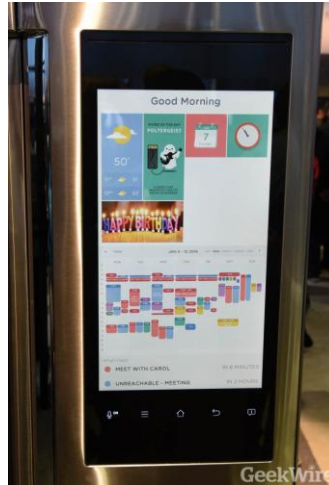
Software Testing & Debugging

The material in this video is subject to the copyright of the owners of the material and is being provided for educational purposes under rules of fair use for registered students in this course only. No additional copies of the copyrighted work may be made or distributed.

GUI Testing

► Graphical User Interface:

- ❖ where all the functionality comes together
- ❖ Interacts with the underlying code



GUI Testing

► (Functional) GUI Testing:

- ❖ Many applications come with a GUI front-end:
 - computers, phones, tablets, medical devices, cars, refrigerators etc.
- ❖ GUI is the user's perspective; the topmost layer via which user interacts with the application
- ❖ GUI Testing = Testing the application from user's perspective:
 - detecting real failures (both GUI and business logic ones)

► Non-functional GUI Testing:

- ❖ Usability testing

OpenCV GUI Error Handler

Incorrect size of input array (Non-positive cols or rows)
in function cvInitMatHeader, C:\User\VP\opencv\cxcore\src\cxarray.cpp(182)

Press "Abort" to terminate application.
Press "Retry" to debug (if the app is running under debug).
Press "Ignore" to continue (this is not safe).

Abbrechen

Wiederholen

Ignorieren

DATAFLEX CRITICAL ERROR

Cannot Find TERMLIST.CFG

OK

Unhandled Access Violation Reading 0x0000X
Exception At 60c068h.
Net Framework 2.0 missing library.

ABORT

CHECK FOR UPDATES

Windows Internet Explorer

Configuration change failed. Please report the following error: Parser return code failed.

OK

The application failed to initialize because the window station is
being shut down.

OK

Jape

About Jape

Services

Hide Jape ⌘H

Hide Others ⌘⇧H

Show All

Quit Jape ⌘Q

GUI error

Dispatcher sees null line -- is jape_engine ok?

Continue

Quit

VirtualBox - Error

Failed to get the access
registered media may b

Caller RC:

VirtualBox - Error

Failed to get the accessibility state of the media . Some of the
registered media may become inaccessible.

Caller RC: 0x80004002

OK

OK

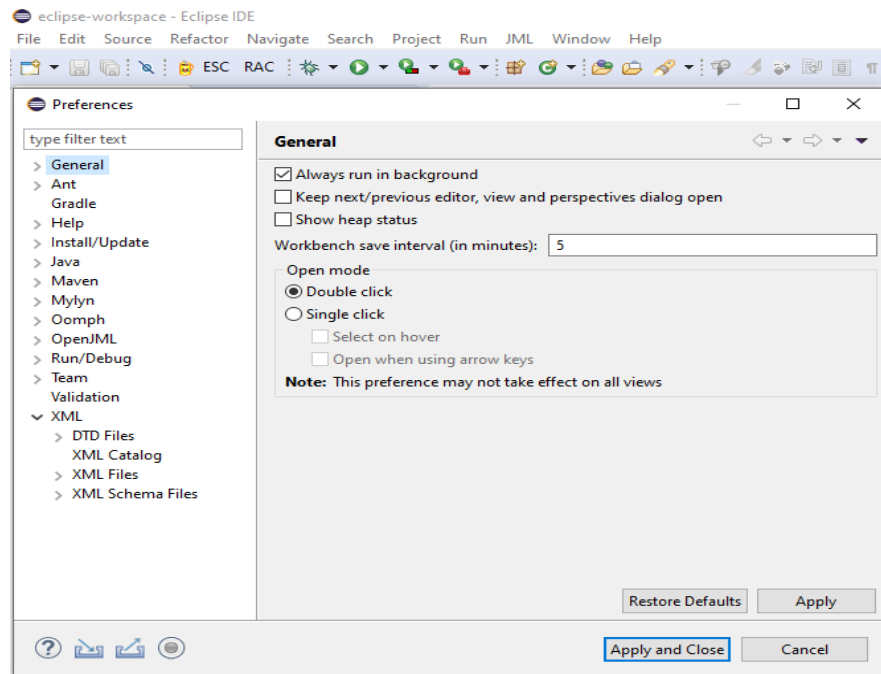
GUI

► **GUI:** A set of visual manipulable objects called controls, aka *widgets or elements*

❖ Each widgets has:

1. a set of properties P where each property $p \in P$ has a value V at any point of time during execution

❖ Example: widget $w = \text{button}$ has property $\text{width} = 120$ and a number of events for instance *pressed*



GUI

► **GUI:** A set of visual manipulable objects called controls, aka *widgets or elements*

❖ **Example: The “Apply and Close” button:**
❖ **properties:**

- **Appearance:**

- Text = “Apply and Close”
- background color = #D3D3D3
- font size = 8

AND MORE APPEARANCE PROPERTIES...

- **Dimension/Position:**

- Height = 17
- Width = 86
- X/Y Coordinates = 2021, 1854

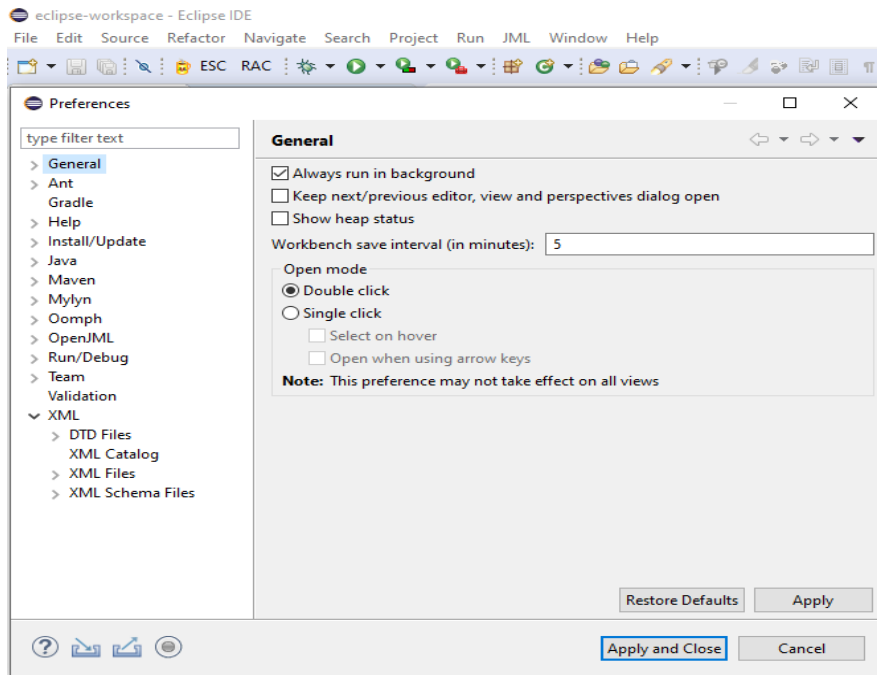
AND MORE Dimension/Position PROPERTIES...

- **Behavior/State:**

- Visibility = true
- Enabled = true

AND MORE Behavior/State Properties ...

- **AND MORE Categories of Properties ...**



GUI

► **GUI:** A set of visual manipulable objects called controls, aka *widgets or elements*

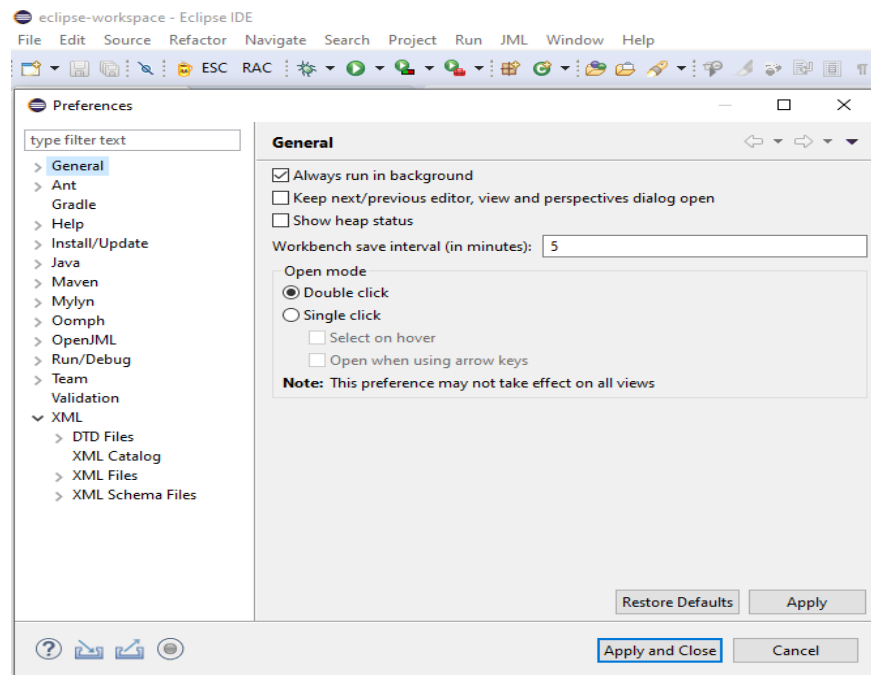
❖ Example:

1. The “Apply and Close” button:

- **Events (most important ones for a button):**
 - **MouseDown** (pressing), **MouseUp** (releasing), **MouseEnter/MouseLeave** (hovering), and **Click** (full interaction)

2. The text-box:

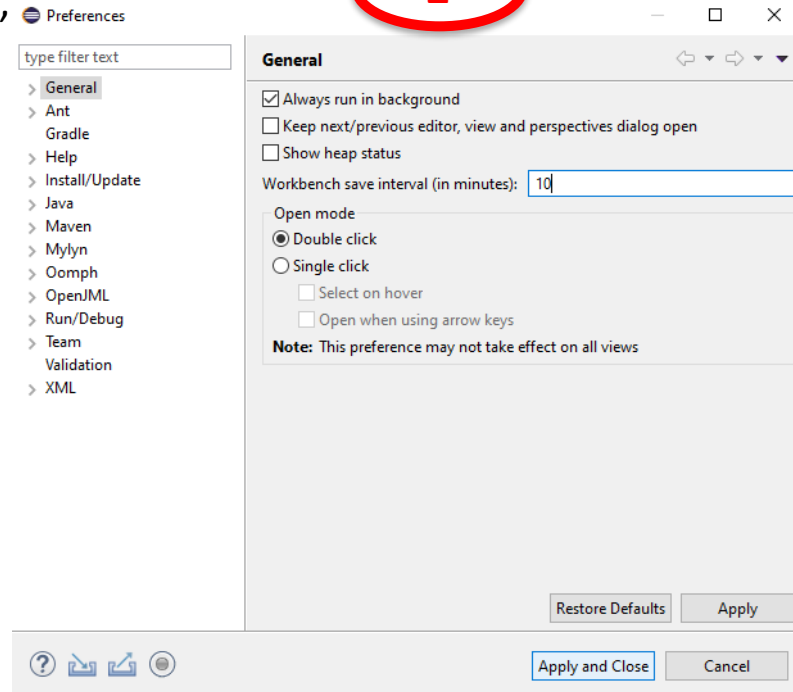
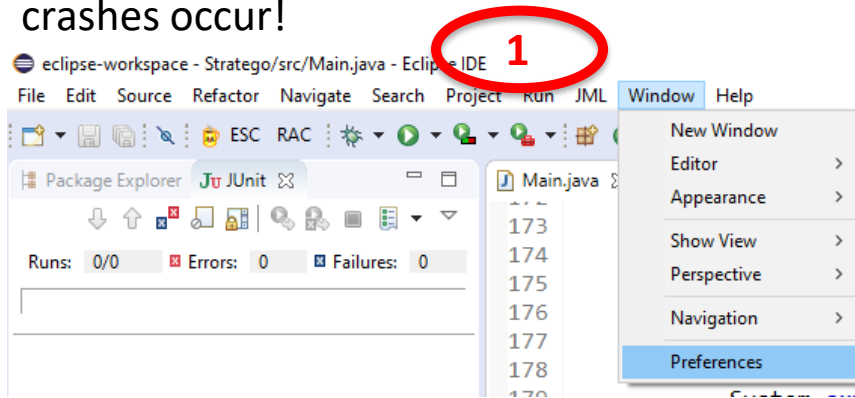
- **Events (most important ones for a textbox):**
 - **TextChanged:** Fires whenever the content changes, either by user input or programmatically.
 - **FocusIn / GotFocus:** Triggered when the user clicks inside or tabs into the text box.
 - **FocusOut / LostFocus:** Triggers when the user clicks outside or leaves the text box.
 - **Click / Double-Click:** Detects mouse interaction within the text box.



GUI Test Case

► Test Case: a sequence of **events** to be run consecutively on the GUI AND Verify results:

- ❖ Example: **Click** “Window” menu, **Click** “Preferences” menu item, **Click** “General”, **Check ON** “Always run in background” checkbox, **Type Text** “10” in “workbench save interval” textbox, **Click** “Apply and close”, Verify no exception or crashes occur!



GUI Testing

- ▶ Three approaches to GUI testing
 - ❖ **Manual testing**
 - ❖ **Capture and replay**
 - ❖ **Model-based testing**

Manual Testing

▶ A human should interact with all different parts of the GUI to make sure the software works as expected:

❖ Pros:

- human is expert and has the domain knowledge
- human acts as an oracle

❖ Cons:

- Tedious
- error-prone
- slow
- costly

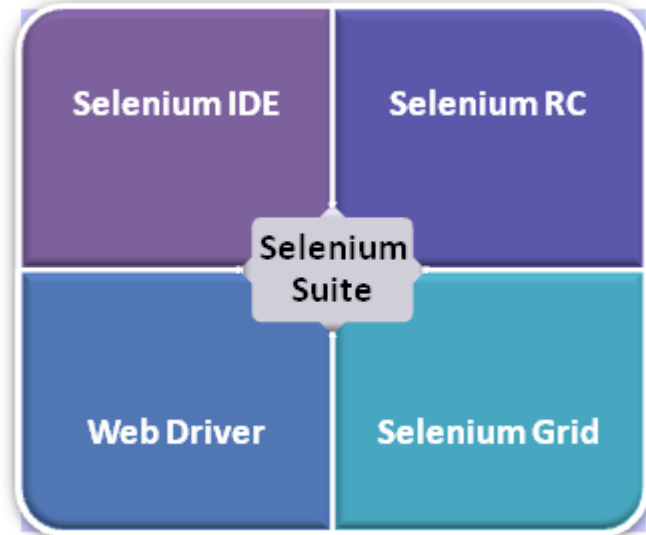


Capture-then-Replay

- ▶ **Capture** (i.e., record) the interactions of a user with the GUI
- ▶ **Replay** (automatically) the recorded sessions over and over
- ▶ Many capture-then-replay tools available:
 - ❖ **Free Software:** Selenium, Abbot, Watir, Autolt etc.
 - ❖ **Commercial:** IBM Robot Tester, Sahi, QF-Test etc.

Selenium

- ▶ An open-source GUI testing tool for web applications
 - ❖ **Selenium IDE:** capture-then-replay (mostly legacy now!)
 - ❖ **Web Driver:** automated test script

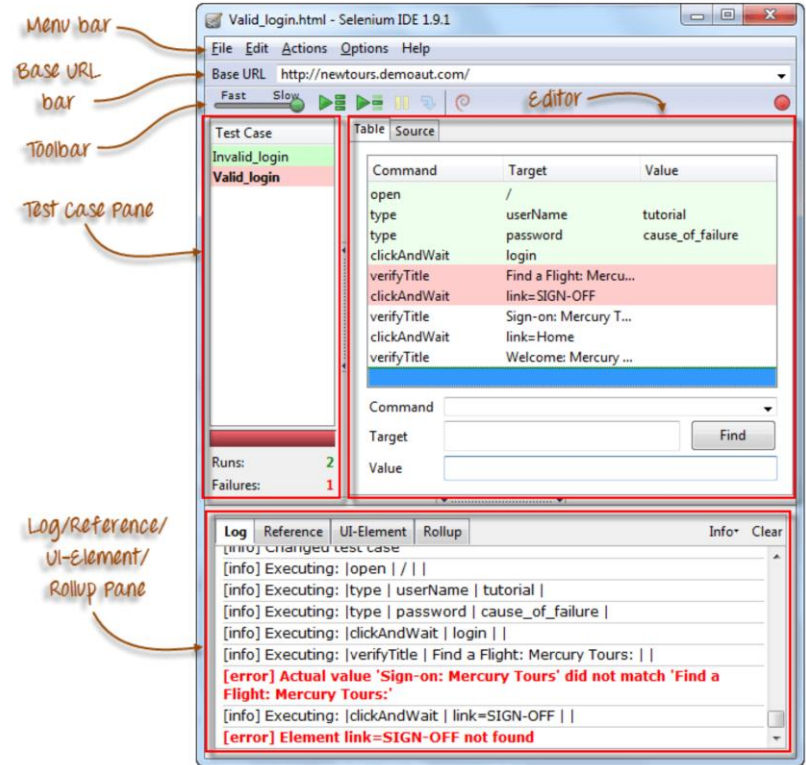


Why the name “Selenium”?

- ▶ Came from a joke
- ▶ Another automated testing framework was popular at the time by Mercury Interactive (later was acquired by HP)
- ▶ “Selenium” is a well-known antidote for Mercury poisoning

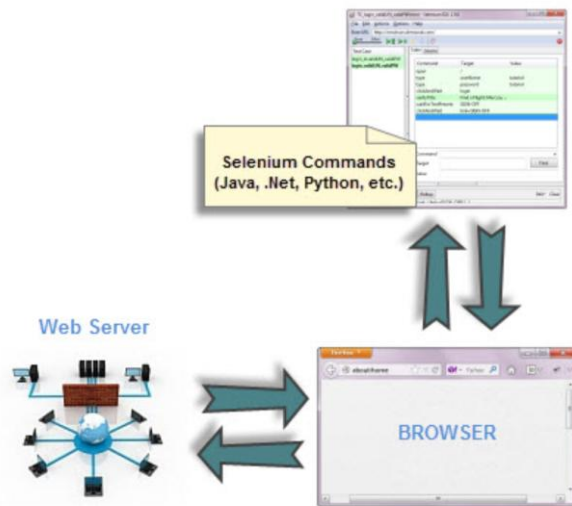
Selenium IDE

- ▶ Available as plug-in (i.e., add-on) for different browsers
- ▶ No longer works on Chrome, but works on Firefox
- ▶ Can record a session and then replay it back
- ▶ Usually used as a prototyping tool. If you want to create more advanced test cases, you will need to use either WebDriver.



WebDriver

- ▶ Write tests in different programming languages (Java, Python, Ruby, PHP, etc.)
- ▶ Run tests on different browsers



Demo of Selenium WebDriver

Selenium Locators

- ▶ By CSS ID: `$(byId(...))`
- ▶ By name attribute: `$(byName(...))`
- ▶ By CSS class name: `$(byClassName(...))`
- ▶ By DOM structure or xpath: `$x(...)`
- ▶ By link text: **`$(byLinkText(...))`**
- ▶ By partial link text: **`$(byPartialLinkText(...))`**
- ▶ By HTML tag name: **`$(byTagName(...))`**
- ▶ By HTML name attribute: **`$(byName(...))`**

Page Object Model: An Object-Oriented Design Pattern

- ▶ **Page Object Model** is a way to organize your web UI tests so they are **cleaner, easier to read, and easier to fix** when the website changes.
- ▶ **CORE IDEA:** Each web *page* in the app gets its own class that knows how to interact with that page.

Page Object Model: An Object-Oriented Design Pattern

► Think of it like this 🍴

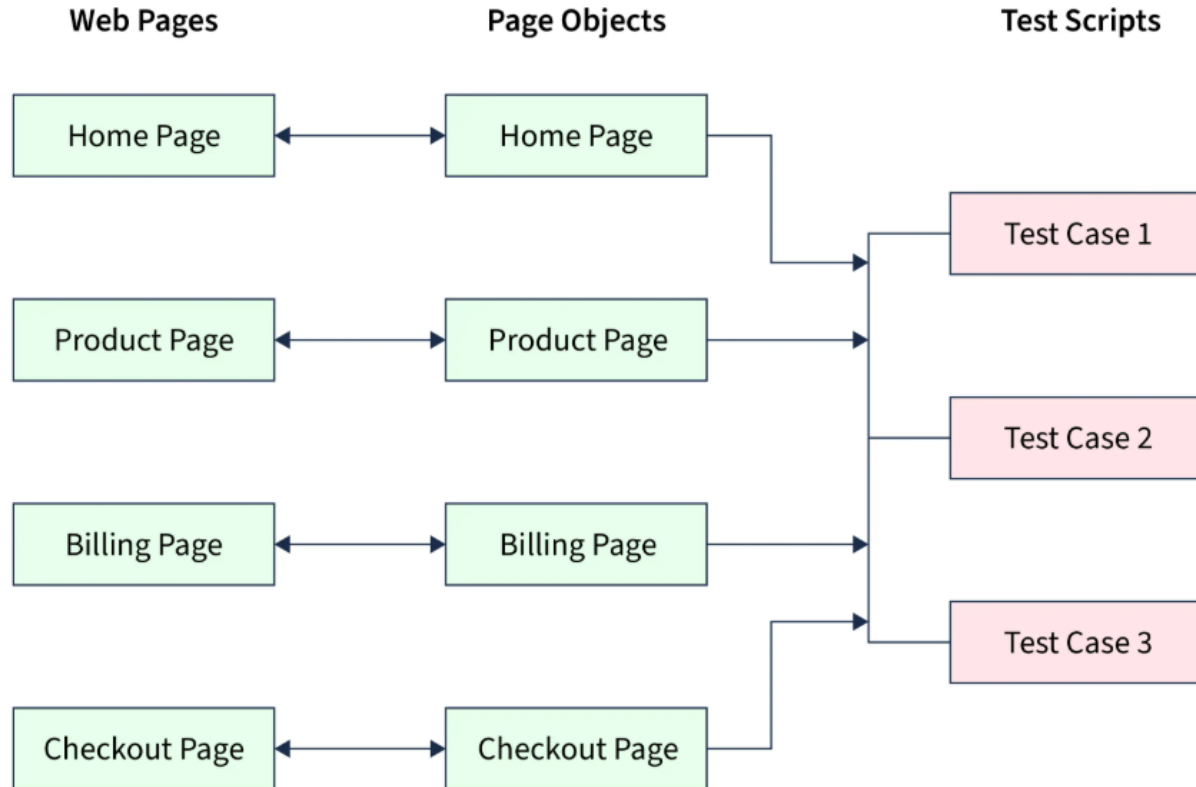
Imagine a **restaurant menu**:

- ❖ You don't go into the kitchen and cook yourself
- ❖ You just say: *"I want a burger"*

► In POM:

- ❖ The **test** says: `loginPage.login("user", "pass")`
- ❖ The **page object (the page class)** handles:
 - Where the *username* field is and type "user"
 - Where the *password* field is and type "pass"
 - Click *submit button*

Page Object Model: An Object-Oriented Design Pattern



Important Notes and Reads

► When Selecting elements using locators:

1. Use `id` if there is a unique one
2. Use `name` if there is a unique one
3. If none of the above, use css or xpath selector (these are fragile though)
4. Avoid selecting by `tag` as much as possible (because most likely results in selecting more than one distinct element!)

► Page Object Model (POM) in Selenium:

- ❖ https://www.selenium.dev/documentation/test_practices/encouraged/page_object_models/



JOHNS HOPKINS
WHITING SCHOOL
of ENGINEERING